

REINFORCEMENT LEARNING

Foundations, Terminology, and the Tools that Power Learning Agents

An Introduction to How Agents Learn by Interacting with Their Environment

What We Will Cover

1

Definitions & Basics

What reinforcement learning is, and how it differs from other ML paradigms

2

Core Terminology

Agent, environment, state, action, reward, policy, value, and more

3

Mathematical Foundations

The Markov Decision Process, returns, and the Bellman equation

4

Types & Algorithms

Model-based vs. model-free, value-based vs. policy-based, and key algorithms

5

Importance & Applications

Why RL matters, where it is used, and the challenges it faces

6

Popular Tools

The frameworks practitioners use to build and train RL agents

Where Reinforcement Learning Fits

Machine learning is broadly organized into three paradigms, distinguished by the kind of feedback the learner receives.



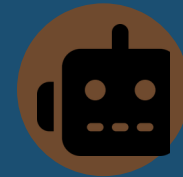
Supervised Learning

Learns from labeled input-output pairs provided up front.



Unsupervised Learning

Finds hidden structure or patterns in unlabeled data.



Reinforcement Learning

Learns by taking actions and receiving reward signals through trial and error.

What Is Reinforcement Learning?

*Reinforcement learning is the branch of machine learning in which an **agent** learns to make good decisions by performing **actions** within an **environment** and receiving **rewards** or penalties in return, gradually improving its behavior over time.*

Definition adapted from GeeksforGeeks, "What is Reinforcement Learning?"

In plain terms: the agent is not told which actions to take. Instead, it must discover which actions yield the most reward by trying them and learning from the outcome — much like how humans and animals learn from experience.

GOAL

Maximize cumulative reward

METHOD

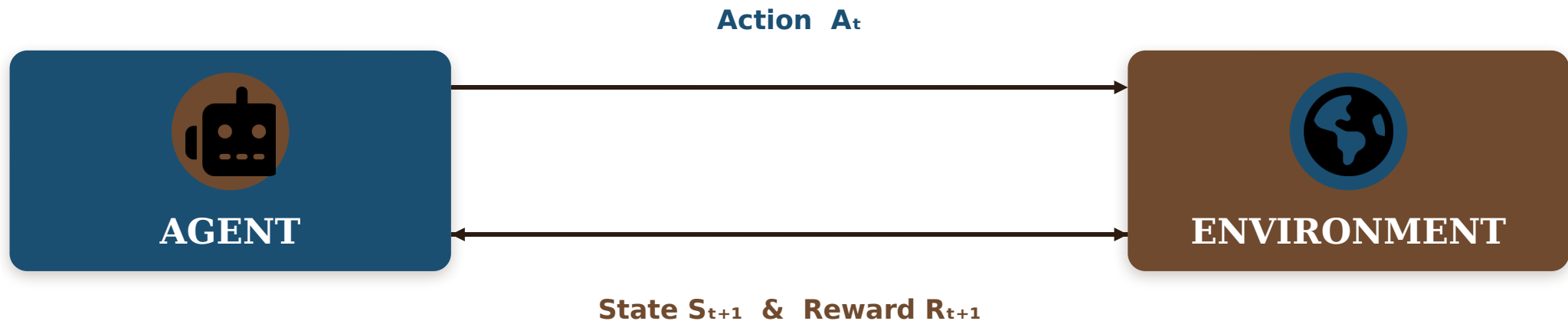
Trial-and-error interaction

FEEDBACK

Delayed, evaluative reward

The Agent-Environment Interaction Loop

At every time step t , the agent observes a state, takes an action, and receives a reward and a new state in return.



The loop repeats: the agent selects an action based on the current state; the environment transitions to a new state and returns a reward signal; the agent updates its behavior to earn more reward over time. This continuous cycle is the foundation of every RL algorithm.

Basic Terminology — Part 1

The four building blocks of every reinforcement learning problem.



Agent

The learner or decision-maker that observes the environment and chooses actions.



Environment

Everything the agent interacts with; it responds to actions with a new state and a reward.



State (S)

A representation of the environment at a given time step, used by the agent to decide.



Action (A)

A choice the agent can make from the set of all possible moves available to it.

Basic Terminology — Part 2

How agents evaluate outcomes and choose behavior.



Reward (R)

Numerical feedback the environment returns after each action, signaling how good it was.



Policy (π)

The agent's strategy — a mapping from states to actions, deterministic or stochastic.



Value Function (V)

The expected cumulative future reward obtainable starting from a given state.

Basic Terminology — Part 3

Additional concepts used throughout RL theory and practice.



Q-Function (Q)

The expected return of taking a specific action in a specific state, then following the policy.



Model

The agent's internal representation of how the environment behaves (transitions and rewards).



Episode & Return

An episode is one full sequence from start to terminal state; the return is its total discounted reward.

The Markov Decision Process (MDP)

RL problems are formally modeled as a Markov Decision Process — a mathematical framework for sequential decision-making.

$$M = (S, A, P, R, \gamma)$$

S

Set of all possible states

A

Set of all possible actions

P

Transition probability, $P(s' | s, a)$

R

Reward function, $R(s, a)$

γ

Discount factor, $0 \leq \gamma \leq 1$

Markov property: *the future depends only on the current state, not on the sequence of events that preceded it.*

Cumulative Reward: The Return

The agent's objective is to maximize the discounted sum of future rewards, called the return.

$$G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \dots$$

Discounted return at time t

$$G_t = \sum \gamma^k R_{t+k+1} \quad (k = 0 \rightarrow \infty)$$

Compact summation form

The Discount Factor γ (gamma)

γ close to 0

The agent is myopic — it values immediate rewards far more than future ones.

γ close to 1

The agent is farsighted — it weighs future rewards nearly as much as immediate ones.

Value Functions & the Bellman Equation

Value functions estimate how good a state (or state-action pair) is; the Bellman equation expresses them recursively.

$$V_{\pi}(s) = E_{\pi}[G_t \mid S_t = s]$$

State-value function under policy π

$$Q_{\pi}(s,a) = E_{\pi}[G_t \mid S_t = s, A_t = a]$$

Action-value (Q) function under policy π

Bellman Equation

$$V_{\pi}(s) = \sum_a \pi(a|s) \sum_{s'} P(s'|s,a) [R(s,a,s') + \gamma V_{\pi}(s')]$$

Key insight: the value of a state equals the expected immediate reward plus the discounted value of the next state — this recursive relationship underlies nearly every RL algorithm, including Q-learning.

Model-Based vs. Model-Free RL

RL approaches differ in whether the agent learns and uses a model of the environment.



Model-Based

- Learns or is given a model of transitions $P(s'|s,a)$ and rewards
- Can plan ahead by simulating future outcomes
- Sample-efficient, but the model may be imperfect
- Example: Dyna-Q, AlphaZero



Model-Free

- Learns a policy or value function directly from experience
- No explicit model of the environment is built
- Simpler to implement; needs more interaction data
- Example: Q-learning, DQN, PPO

Value-Based, Policy-Based & Actor-Critic

Among model-free methods, algorithms differ in exactly what they learn.



Value-Based

Learns a value/Q-function and derives a policy by acting greedily with respect to it. Example: Q-learning, DQN.



Policy-Based

Directly optimizes the policy $\pi(a|s)$ using gradient ascent on expected return. Example: REINFORCE.



Actor-Critic

Combines both: an actor updates the policy while a critic estimates value to reduce variance. Example: PPO, A3C.

Exploration vs. Exploitation

One of the central dilemmas in reinforcement learning: should the agent try something new, or stick with what already works?



Exploration

Trying new, uncertain actions to discover potentially better strategies and gather information about the environment.



Exploitation

Choosing the best-known action based on current knowledge to maximize immediate reward.

ϵ -greedy strategy: *with probability ϵ explore a random action, otherwise exploit the current best-known action.*

Why Reinforcement Learning Matters

RL is one of the few paradigms that lets machines learn optimal behavior without being explicitly told what to do.



No Labeled Data Required

Learns purely from interaction and reward signals, unlike supervised learning.



Handles Sequential Decisions

Optimizes long-term outcomes, not just single-step predictions.



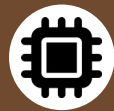
Adapts to Changing Environments

Continuously updates its policy as conditions evolve.



Achieves Superhuman Performance

Has surpassed humans in games such as Go, Chess, and Dota 2.



Enables Autonomous Systems

Powers robotics, self-driving cars, and industrial automation.



Optimizes Complex Objectives

Balances multiple, often competing, long-term goals effectively.

Popular Reinforcement Learning Algorithms

A brief look at four algorithms that shaped modern RL, from tabular methods to deep policy optimization.

1

Q-Learning

$$Q(s,a) \leftarrow Q(s,a) + \alpha [R + \gamma \max_{a'} Q(s',a') - Q(s,a)]$$

Off-policy, model-free algorithm that learns the optimal action-value function directly.

2

Double Q-Learning / DQN

$$y = R + \gamma Q(s', \operatorname{argmax}_{a'} Q(s',a'); \theta); \theta'$$

Uses two value estimators to reduce the overestimation bias of standard Q-learning (van Hasselt, Guez & Silver, 2016).

3

Proximal Policy Optimization (PPO)

$$L(\theta) = E[\min(r_t(\theta) A_t, \operatorname{clip}(r_t(\theta), 1-\epsilon, 1+\epsilon) A_t)]$$

Clips the policy update to stay within a trust region, giving stable, reliable policy-gradient training (Schulman et al., 2017).

Popular Reinforcement Learning Tools

A rich ecosystem of open-source libraries makes it easier to build, train, and benchmark RL agents.



Gymnasium (OpenAI Gym)

Standard toolkit of RL environments — from classic control to Atari — with a common API.



Stable-Baselines3

Reliable PyTorch implementations of PPO, DQN, SAC, and other core algorithms.



Ray RLlib

Scalable, distributed RL library for training agents across clusters of machines.



PyTorch / TensorFlow

Deep learning frameworks used to build the neural networks behind modern RL agents.



Unity ML-Agents

Connects RL agents to rich 3D simulation environments for training and testing.



DeepMind OpenSpiel

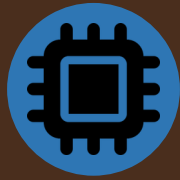
A collection of environments and algorithms for research in games and multi-agent RL.

Where Reinforcement Learning Is Used



Games

Mastering Chess, Go, and video games at superhuman levels (AlphaGo, OpenAI Five).



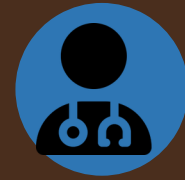
Robotics

Learning locomotion, grasping, and manipulation through trial and error.



Finance

Optimizing trading strategies and portfolio management under uncertainty.



Healthcare

Personalizing treatment plans and optimizing clinical decision sequences.

Challenges & Limitations of RL

Despite its power, reinforcement learning still faces significant practical obstacles.



Sample Inefficiency

Many algorithms require millions of interactions to learn a good policy.



Reward Design

Poorly designed reward functions can lead to unintended or unsafe behavior.



Exploration Difficulty

Sparse or delayed rewards make it hard to discover good strategies.



Stability & Reproducibility

Training can be sensitive to hyperparameters and random seeds.

Key Takeaways

- 1 RL is learning through interaction: agents choose actions and learn from reward signals, not labeled data.
- 2 The agent-environment loop, states, actions, rewards, and policies form the vocabulary of every RL system.
- 3 The Markov Decision Process and the Bellman equation provide the mathematical backbone of RL.
- 4 Algorithms range from tabular Q-learning to deep methods like DQN and PPO.
- 5 Frameworks such as Gymnasium, Stable-Baselines3, and RLlib make RL accessible and reproducible.

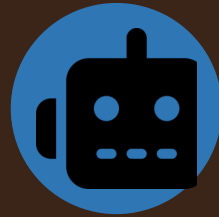
References

1 GeeksforGeeks. What is Reinforcement Learning? Retrieved from [geeksforgeeks.org/machine-learning/what-is-reinforcement-learning/](https://www.geeksforgeeks.org/machine-learning/what-is-reinforcement-learning/)

2 Sutton, R. S., & Barto, A. G. (2019). Reinforcement Learning: An Introduction (2nd ed.). MIT Press.

3 van Hasselt, H., Guez, A., & Silver, D. (2016). Deep Reinforcement Learning with Double Q-Learning. Thirtieth AAAI Conference on Artificial Intelligence.

4 Schulman, J., Wolski, F., Dhariwal, P., Radford, A., & Klimov, O. (2017). Proximal Policy Optimization Algorithms. arXiv preprint arXiv:1707.06347.



Thank You

Questions & Discussion